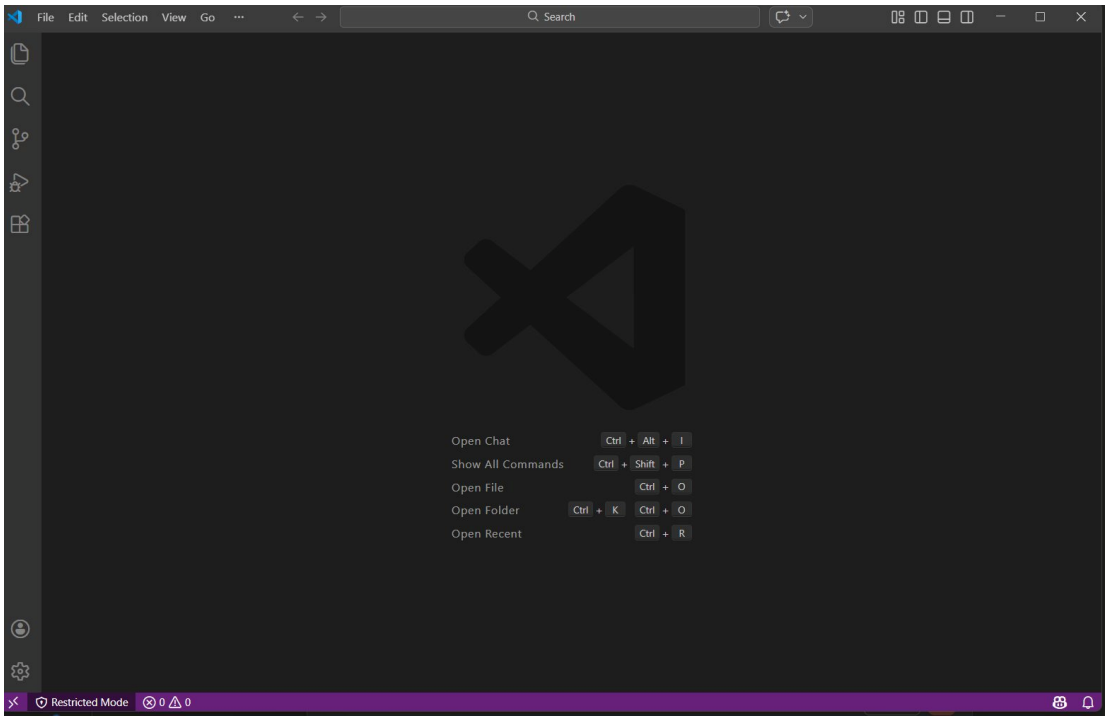
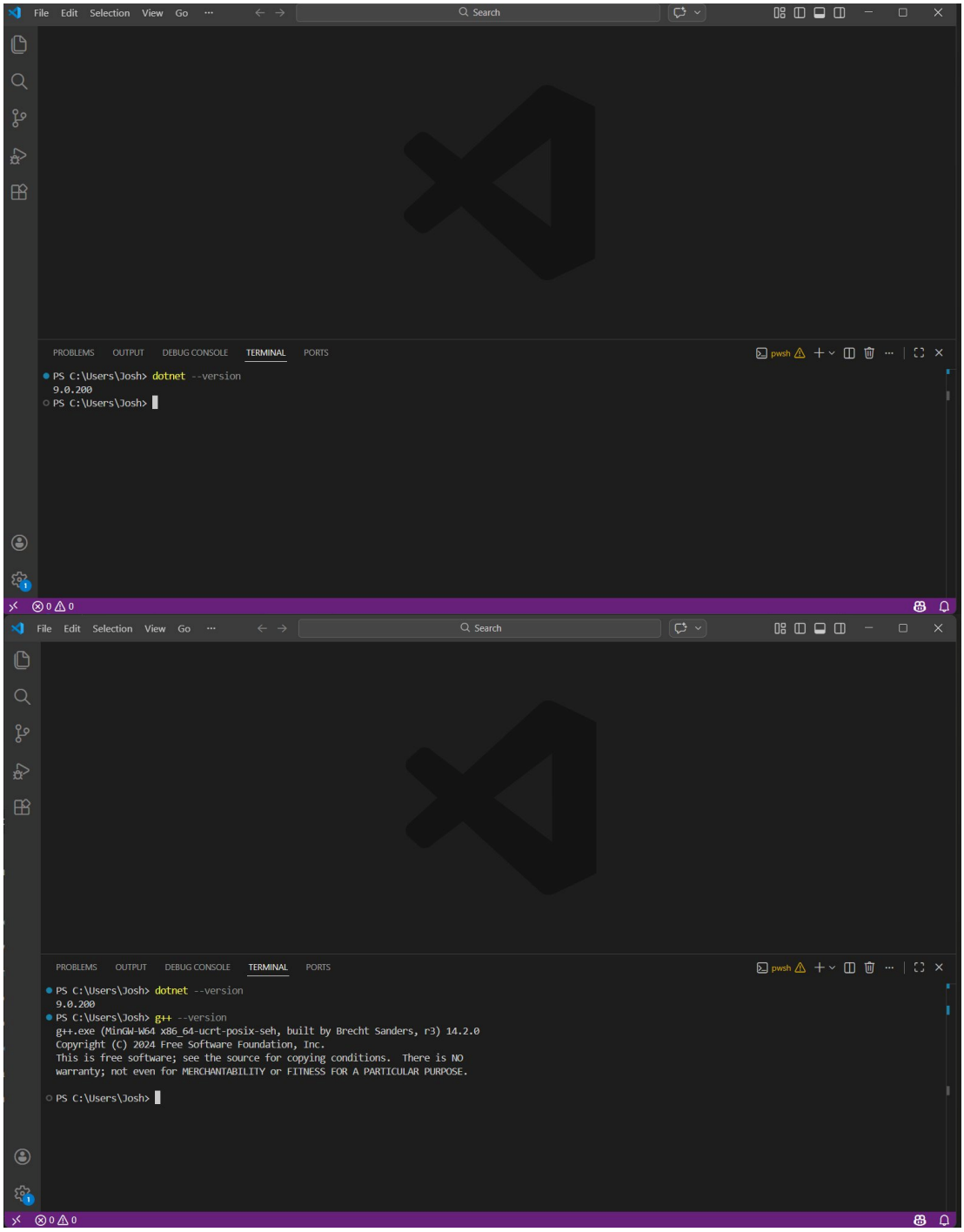
Build and Run

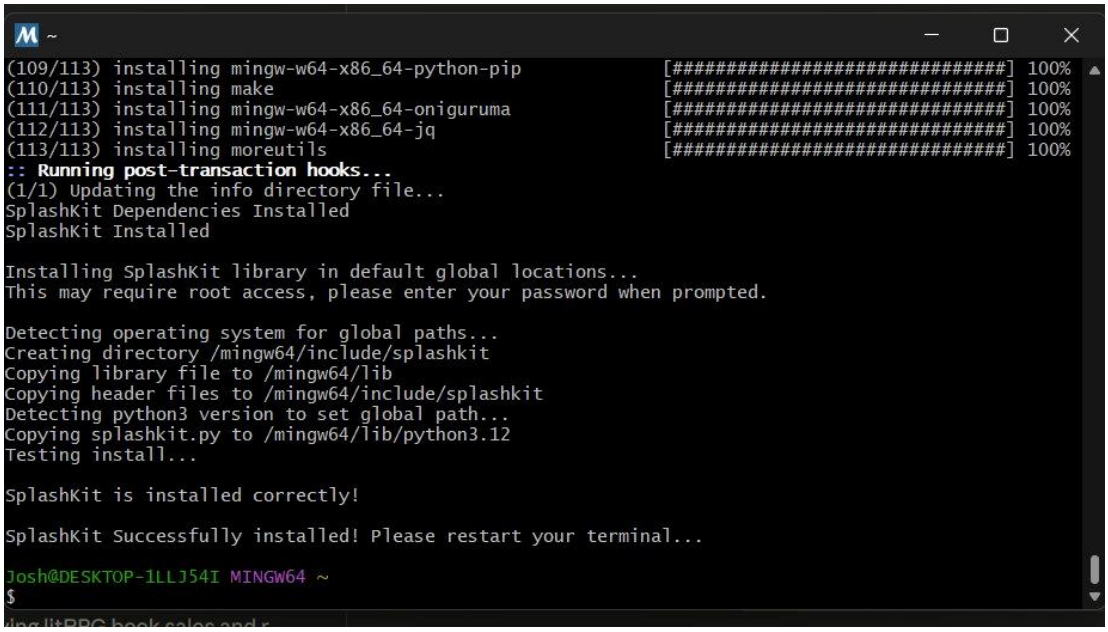
Name: Joshua Pijaca
Student ID: 223422713
Date: 22/03/2026

Learning Journey and Evidence

To learn about build and run, I did the following.

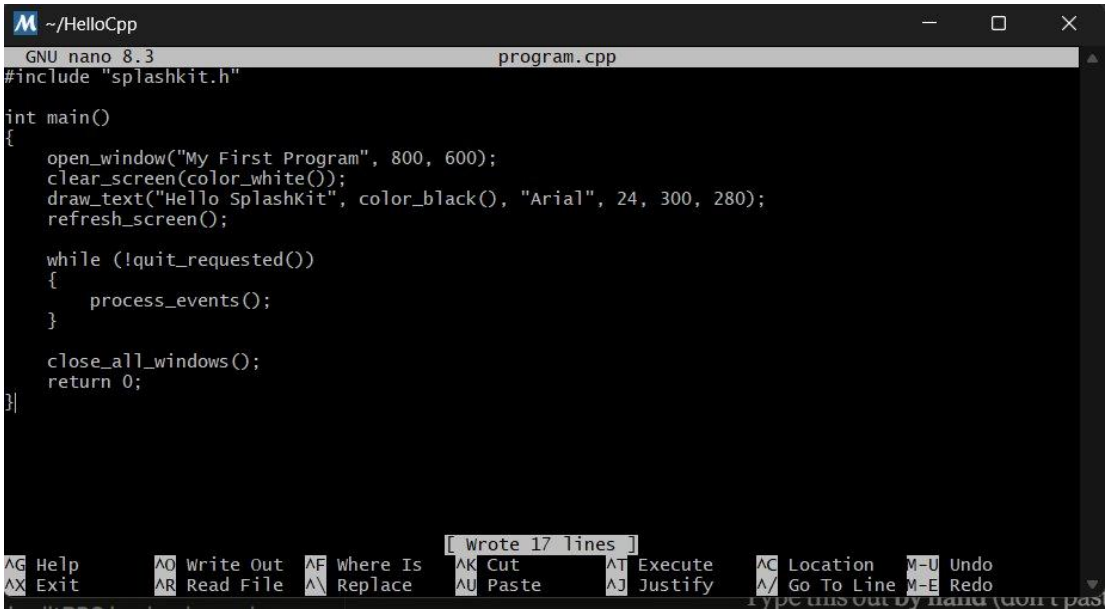
Installed VS Code, .NET SDK 9.0.200, MSYS2 with g++ 14.2.0, and SplashKit


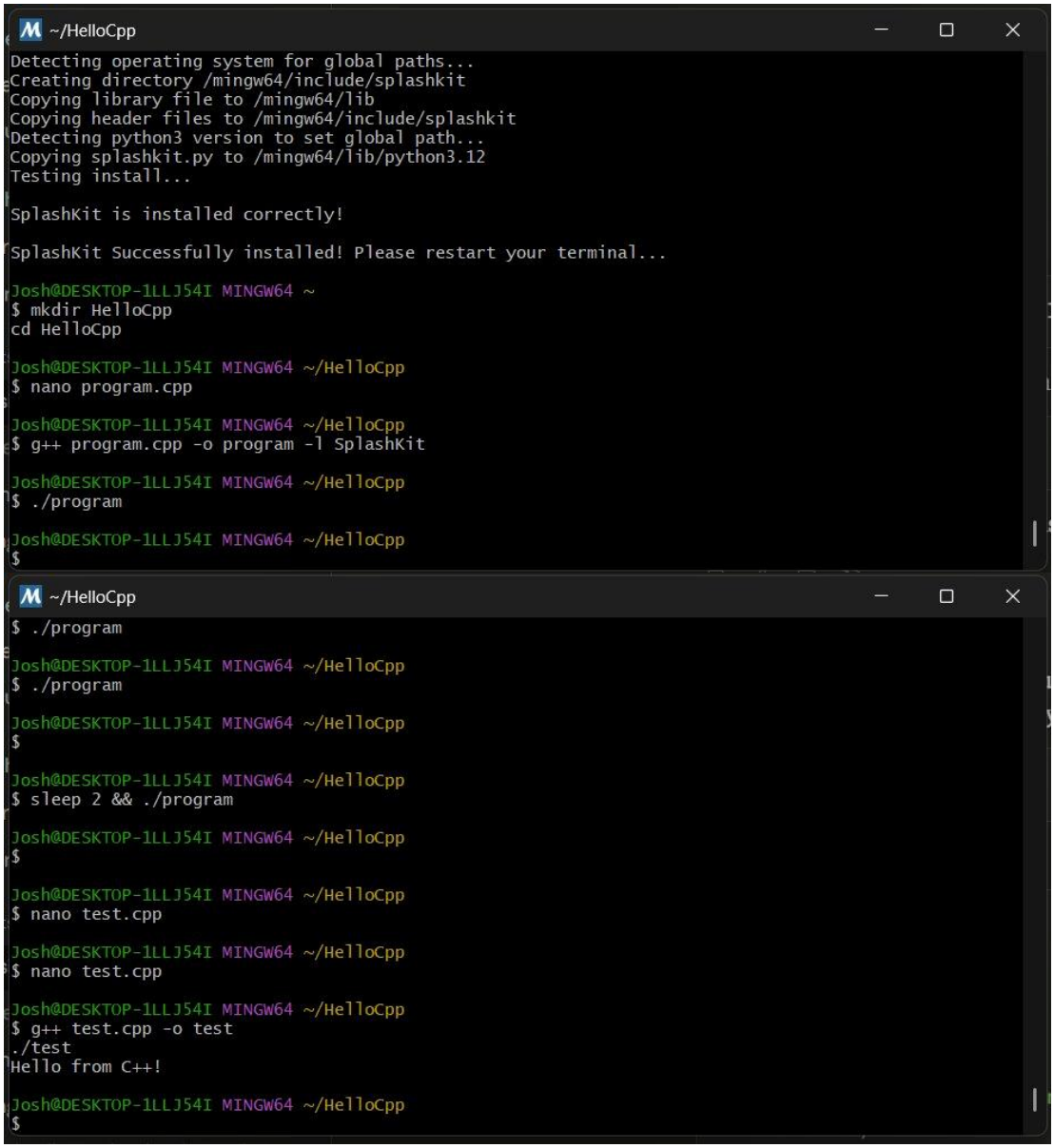

File Edit Selection View Go
Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
pwsh
PS C:\Users\Josh> dotnet --version
9.0.200
PS C:\Users\Josh>
File Edit Selection View Go
Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
pwsh
PS C:\Users\Josh> dotnet --version
9.0.200
PS C:\Users\Josh> g++ --version
g++.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
PS C:\Users\Josh>



Setting up the development environment involved several tools working together. I installed VS Code as my code editor, then the .NET SDK for C# development and MSYS2 with g++ for C/C++ compilation. Installing SplashKit required first installing git in MSYS2 before the install script would work, which was an unexpected extra step. I learnt that different tools are needed for different programming languages, and that getting everything set up correctly is an important first step before you can start coding.

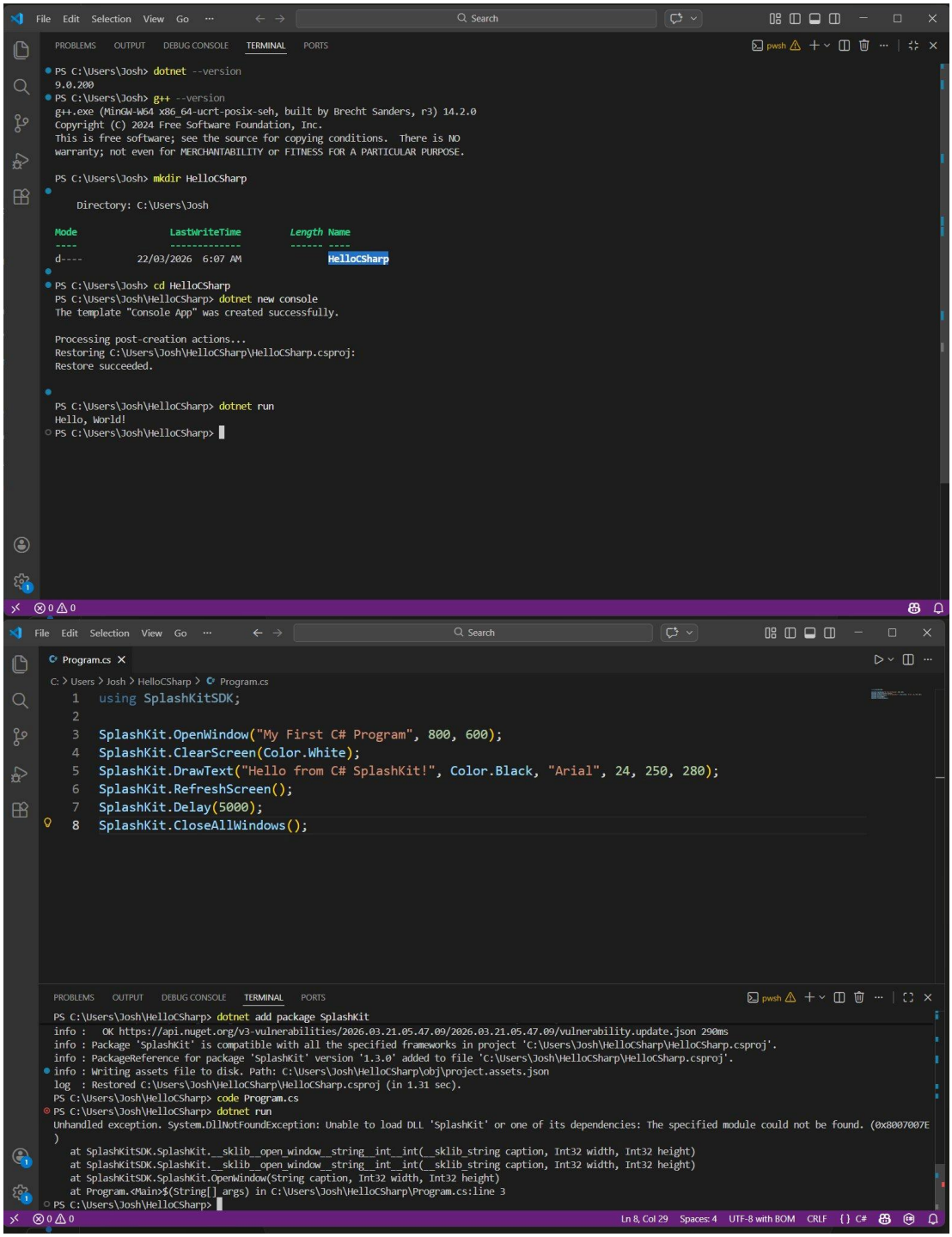
Created, compiled, and ran a C++ program using g++ in the MSYS2 MINGW64 terminal

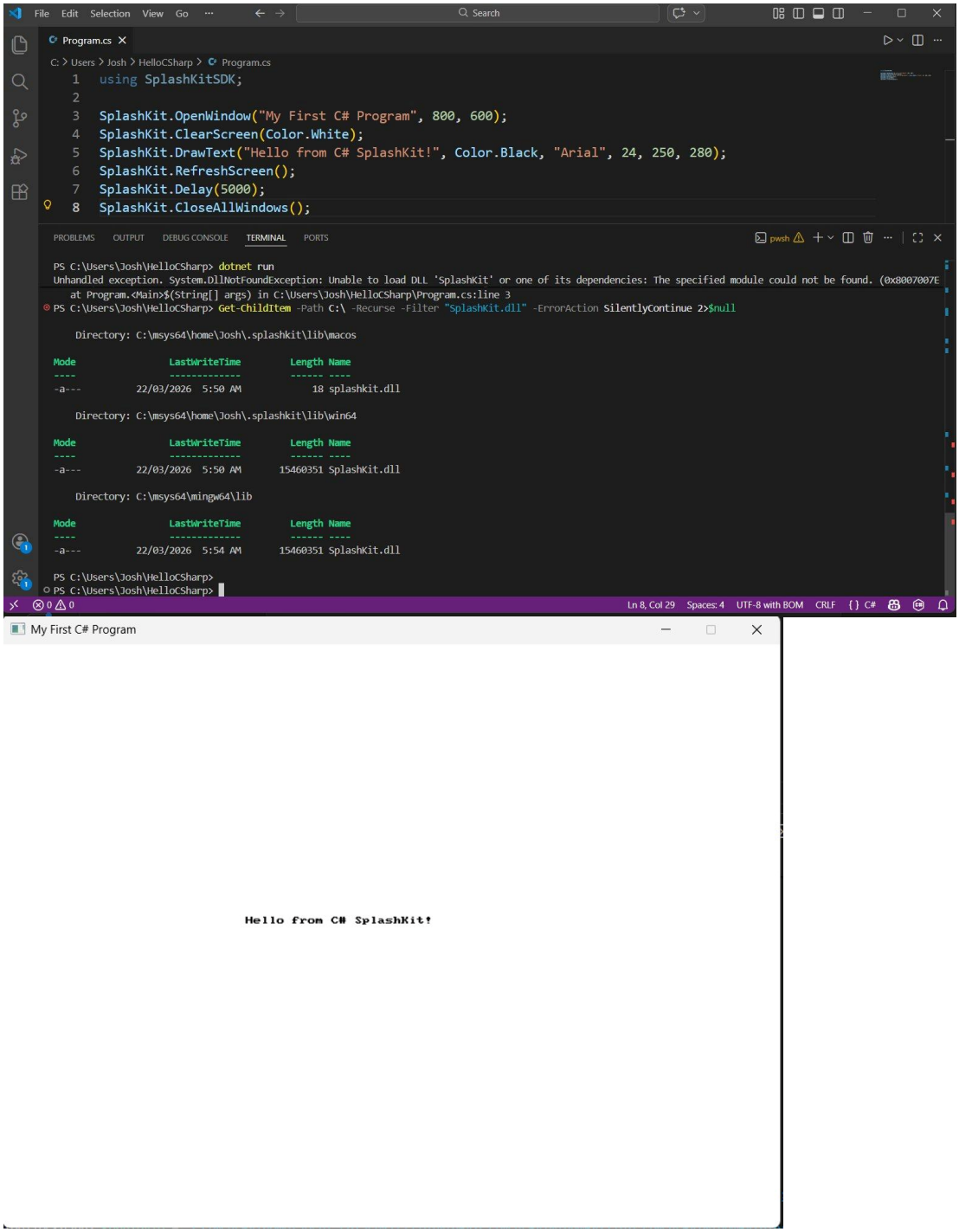




I used the mkdir and cd commands to create and navigate into a project folder, then used nano to write C++ code. I compiled it using g++ with the -l SplashKit flag to link the SplashKit library. My first attempt at pasting code from an external source caused a compilation error due to hidden Unicode characters, which taught me that copying code can introduce invisible problems. I had to retype the code manually to fix it. I also created a simpler test.cpp without SplashKit to confirm g++ was working, which printed "Hello from C++!" successfully.

Created, built, and ran a C# SplashKit project using dotnet in VS Code terminal


File Edit Selection View Go
Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Josh> dotnet --version
9.0.200
PS C:\Users\Josh> g++ --version
g++.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
PS C:\Users\Josh> mkdir HelloCSharp
Directory: C:\Users\Josh
Mode LastWriteTime Length Name
d---- 22/03/2026 6:07 AM HelloCSharp
PS C:\Users\Josh> cd HelloCSharp
PS C:\Users\Josh\HelloCSharp> dotnet new console
The template "Console App" was created successfully.
Processing post-creation actions...
Restoring C:\Users\Josh\HelloCSharp\HelloCSharp.csproj:
Restore succeeded.
PS C:\Users\Josh\HelloCSharp> dotnet run
Hello, World!
PS C:\Users\Josh\HelloCSharp>
File Edit Selection View Go
Search
Program.cs
C: > Users > Josh > HelloCSharp > Program.cs
1 using SplashKitSDK;
2
3 SplashKit.OpenWindow("My First C# Program", 800, 600);
4 SplashKit.ClearScreen(Color.White);
5 SplashKit.DrawText("Hello from C# SplashKit!", Color.Black, "Arial", 24, 250, 280);
6 SplashKit.RefreshScreen();
7 SplashKit.Delay(5000);
8 SplashKit.CloseAllWindows();
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Josh\HelloCSharp> dotnet add package SplashKit
info : OK https://api.nuget.org/v3-vulnerabilities/2026.03.21.05.47.09/2026.03.21.05.47.09/vulnerability.update.json 290ms
info : Package 'SplashKit' is compatible with all the specified frameworks in project 'C:\Users\Josh\HelloCSharp\HelloCSharp.csproj'.
info : PackageReference for package 'SplashKit' version '1.3.0' added to file 'C:\Users\Josh\HelloCSharp\HelloCSharp.csproj'.
info : Writing assets file to disk. Path: C:\Users\Josh\HelloCSharp\obj\project.assets.json
log : Restored C:\Users\Josh\HelloCSharp\HelloCSharp.csproj (in 1.31 sec).
PS C:\Users\Josh\HelloCSharp> code Program.cs
PS C:\Users\Josh\HelloCSharp> dotnet run
Unhandled exception. System.DllNotFoundException: Unable to load DLL 'SplashKit' or one of its dependencies: The specified module could not be found. (0x8007007E
)
at SplashKitSDK.SplashKit.__sklib__open_window__string__int__int(__sklib_string caption, Int32 width, Int32 height)
at SplashKitSDK.SplashKit.__sklib__open_window__string__int__int(__sklib_string caption, Int32 width, Int32 height)
at SplashKitSDK.SplashKit.OpenWindow(String caption, Int32 width, Int32 height)
at Program.<Main>$(String[] args) in C:\Users\Josh\HelloCSharp\Program.cs:line 3
PS C:\Users\Josh\HelloCSharp>
Ln 8, Col 29 Spaces: 4 UTF-8 with BOM CRLF { } C#



I used the dotnet tool to create a new console project and added the SplashKit NuGet package. My first attempt to run the SplashKit program failed with a DllNotFoundException because the SplashKit native library was not in the system PATH. I had to search my computer to find where the SplashKit.dll was located, and then add that path to the environment variable before dotnet run would work. This taught me that programs sometimes depend on external libraries, and the system needs to know where to find them. Once the PATH was set correctly, the program ran and displayed a window saying "Hello from C# SplashKit!".

Brief Summary of Concepts

Concept / Commands	Key Idea / Concept
Shell	A text-based interface that lets you interact with your computer by typing commands. Examples include PowerShell on Windows and Bash in MSYS2. You type instructions and the shell executes them.
Compiler	A tool that translates human-readable source code into machine code that the computer can execute. For example, g++ compiles C++ code, and dotnet build compiles C# code.

Action	Example shell command showing how to do this
Move into a folder	`cd HelloCpp`
Make a folder	`mkdir HelloCpp`
Create a dotnet project	`dotnet new console`
Add SplashKit to the project	`dotnet add package SplashKit`
Build and run a dotnet project	`dotnet run`
Compile a C/C++ program – using SplashKit	`g++ program.cpp -o program -l SplashKit`
Run the C++ program	`./program`

Reflection

What is the most important thing you learned from this task and why?

The most important thing I learned is that programming involves more than just writing code. Before I could write a single line, I had to set up an entire toolchain including a code editor, compilers, and libraries. I also learned that when something goes wrong, the error messages usually point you in the right direction if you read them carefully. For example, the DllNotFoundException in C# told me exactly which library was missing, and the Unicode character error in C++ told me the problem was on line 1 of my file. Learning to read and understand error messages is a skill that will be useful throughout programming.

Using the terminal can feel different to using the graphical user interface (GUI). What are your thoughts about using the terminal versus a GUI, and how do you think it could be useful?

At first, using the terminal felt unfamiliar compared to clicking through folders and menus in a GUI. However, I quickly saw that the terminal can be much faster for certain tasks. Creating a folder, navigating into it, and setting up a project took just a few short commands, whereas doing the same through a GUI would involve multiple clicks and windows. I can also see how the terminal would be useful for automation, since you could save a series of commands in a script and run them all at once. I think becoming comfortable with the terminal is an important skill, even though a GUI is more intuitive at first.